[User Group Italiano Sql Server]

PASS
PROFESSIONAL ASSOCIATION
FOR SQL SERVER

[ADO Best Practices]

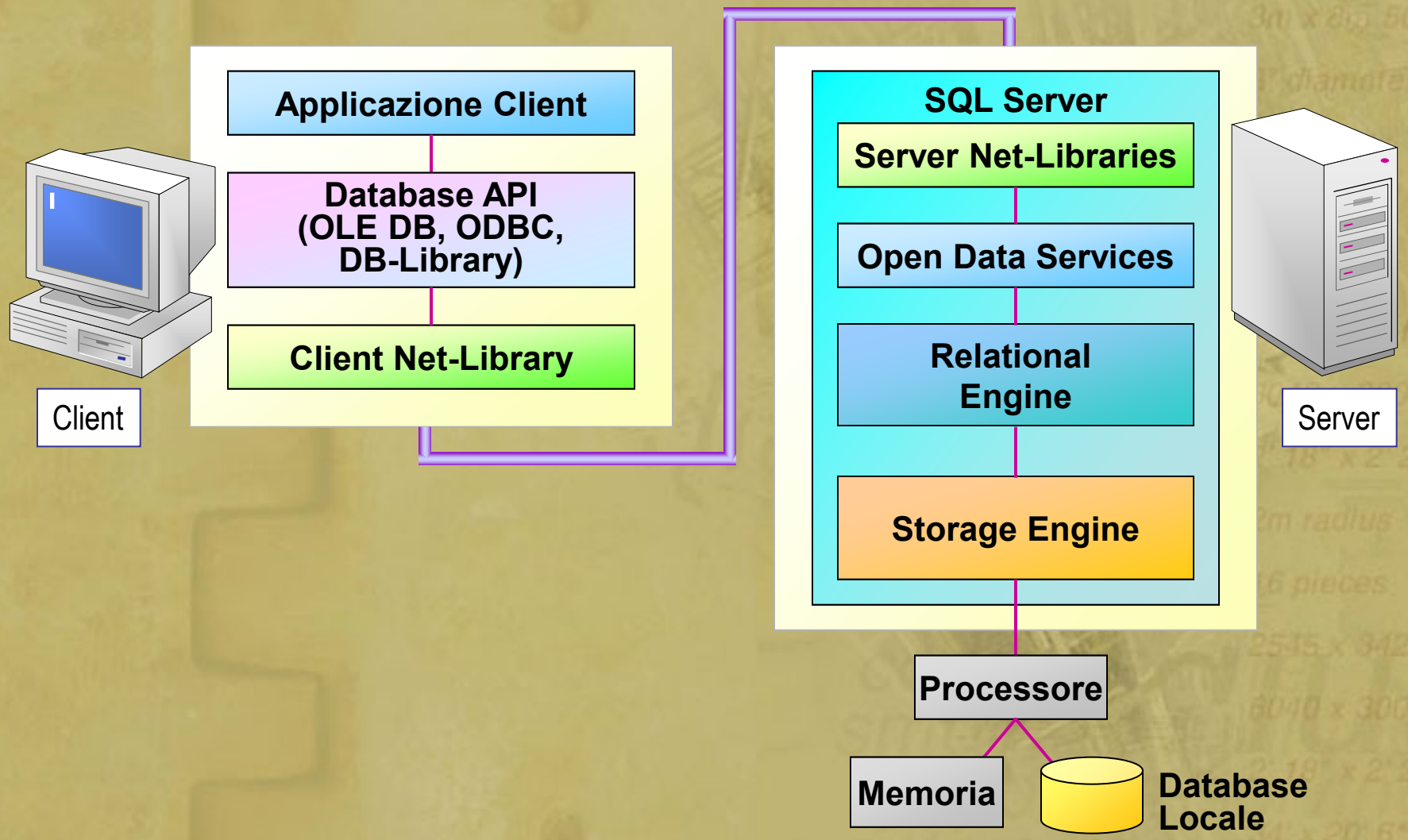
- Franco Perduca (fperduca@ugiss.org)
- Gianluca Hotz (ghotz@ugiss.org)

PASS
PROFESSIONAL ASSOCIATION FOR SQL SERVER

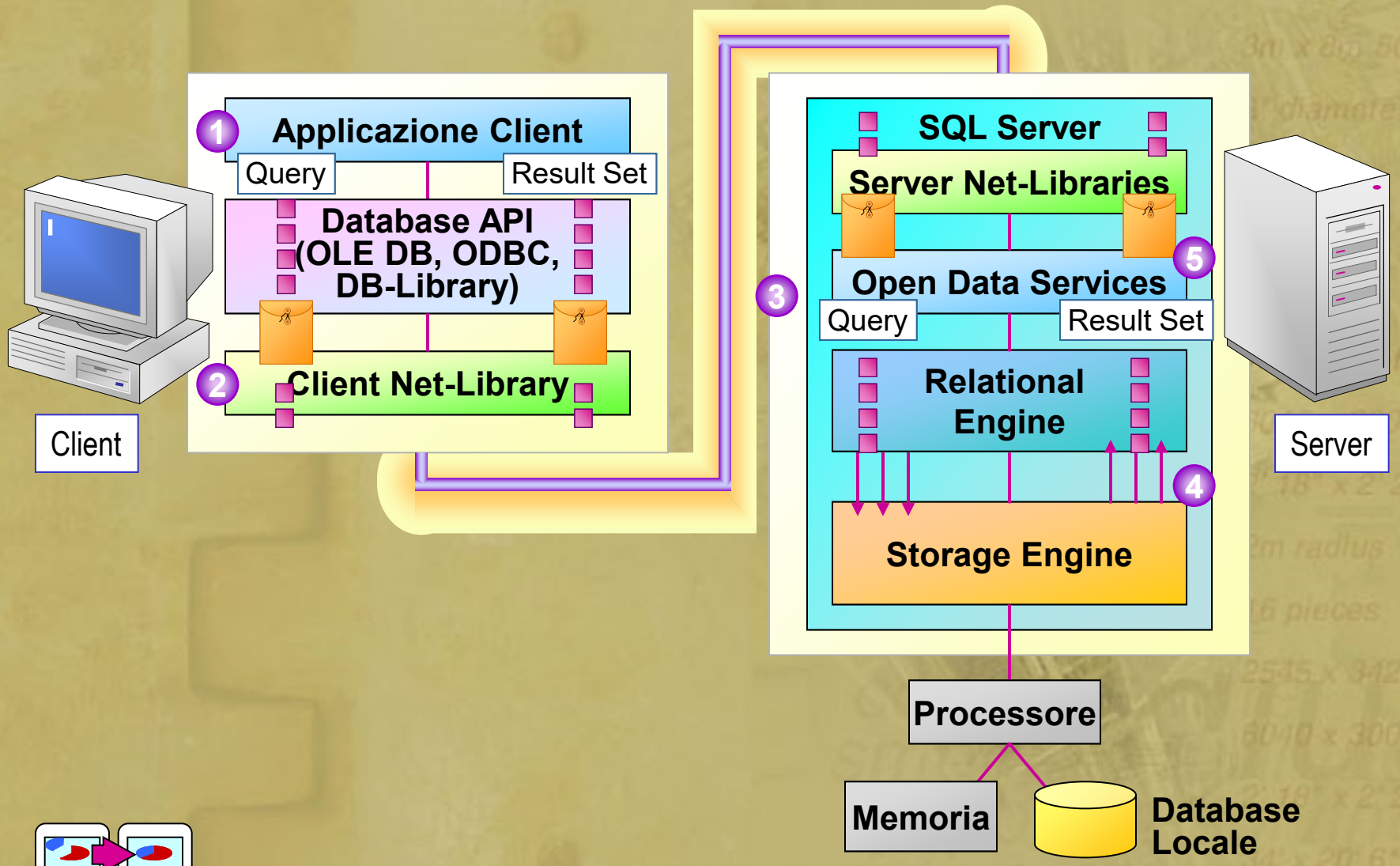
[Agenda]

- Comunicazione client-server
- Introduzione ADO
- Connection pooling
- Query Optimizer
- Accesso ai dati
- Cursori

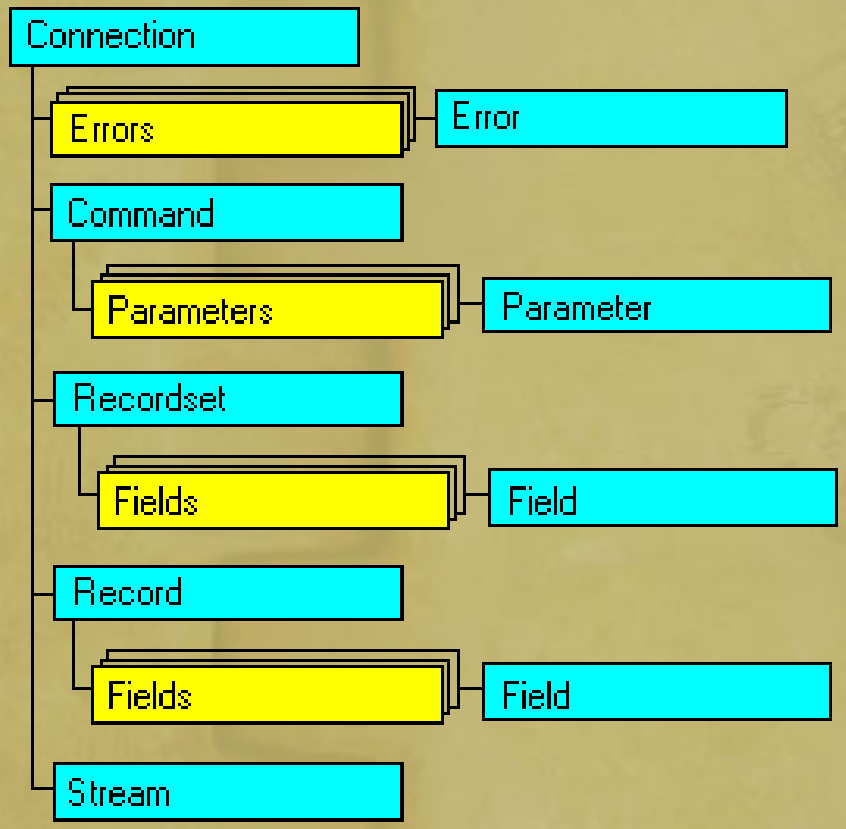
[Componenti Client-Server]



[Comunicazione Client-Server]

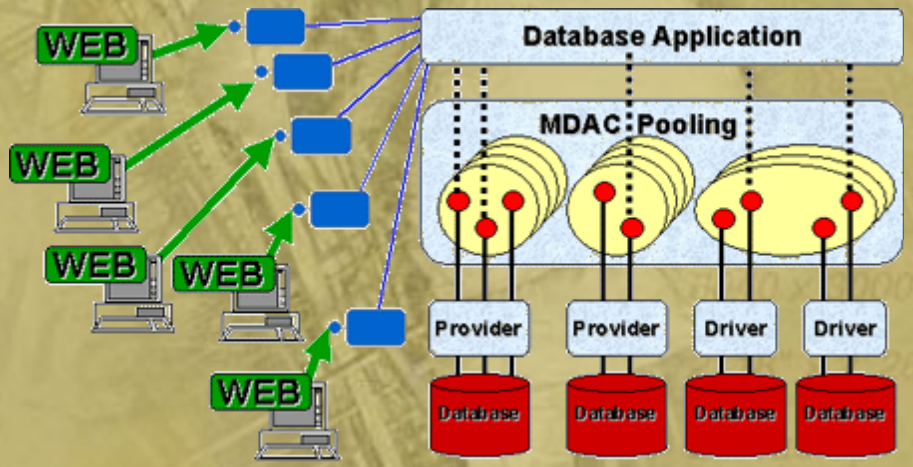


[Introduzione ADO]



[Connection Pooling]

- Tecnica per il riutilizzo delle connessioni
- Gestito dalle librerie sul client
- Sql Server non vede il pool di connessioni
- Per ogni stringa di connessione viene creato un pool
- Sql Server utilizza circa 24 Kb per ogni connessione



[Come si usa]

- Rilascio della connessione al pool :
  - Oggetto COM

```
AdoConn.close
Set AdoConn = nothing
```

  - Applicazione Vb

```
AdoConn.close
```

[Cosa non fare]

- Lasciare aperti oggetti temporanei prima di chiudere la connesione
- Usare gli *application roles*
- Lasciare transazioni aperte

[Tracciare le connessioni]

- Tabelle di sistema
  - `master.dbo.sysprocesses`
  - `master.dbo.sysperfinfo`
- Stored procedure di sistema
  - `sp_who`
  - `sp_who2`

[Tracciare le connessioni]

- Sql Server Profiler
  - Security Audit : Audit Login
  - Security Audit : Audit Logout
- Perfomance Monitor (System Monitor)
  - General Statistic : User Connections
  - General Statistic : Logins/Sec
  - General Statistic : Logouts/Sec

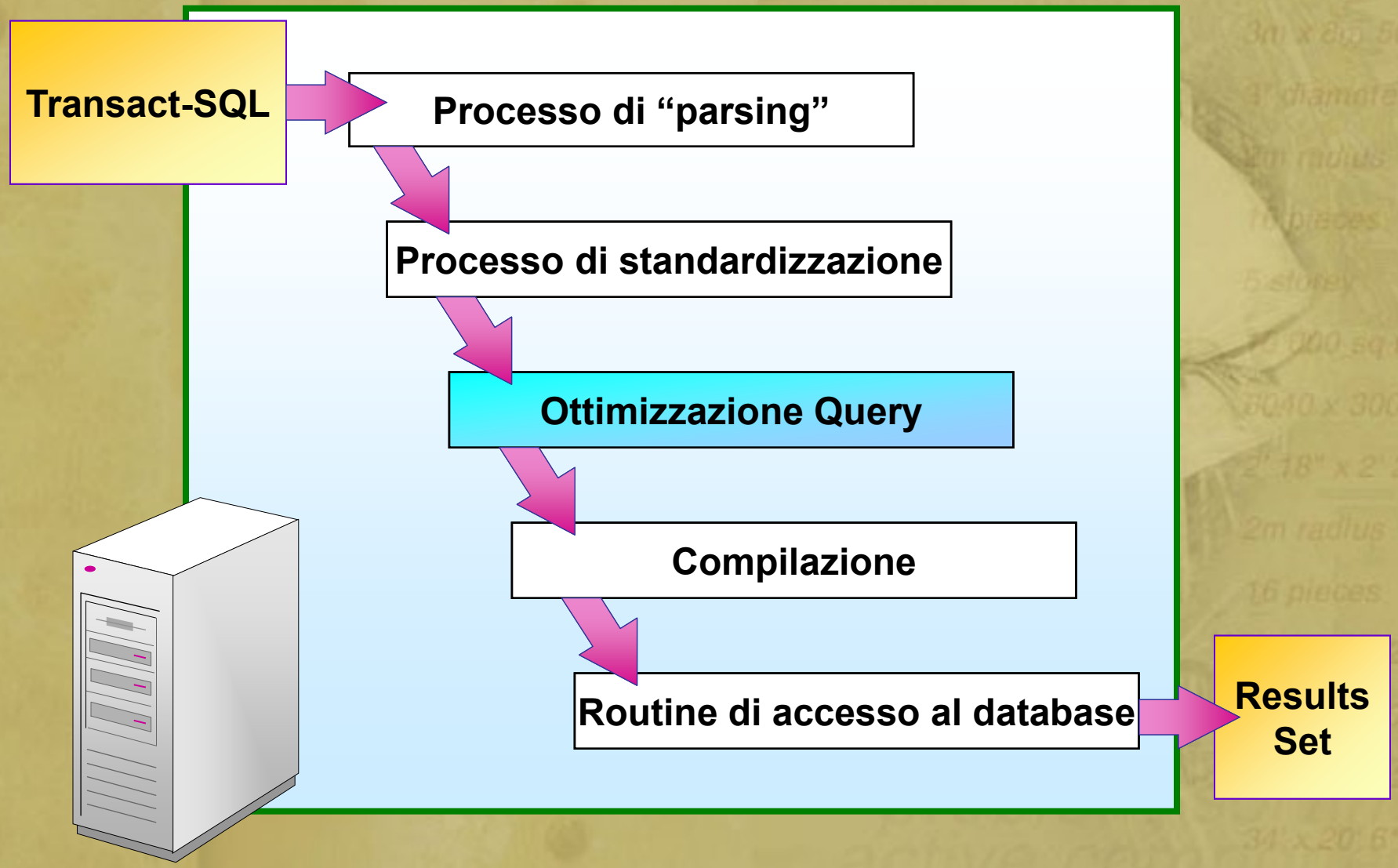
[Accesso ai dati]

- Apertura diretta di una tabella
- Richiesta SQL testuale
- Esecuzione di *Stored Procedures*
- Command e parameters
- Prepared Statements

[Apertura diretta tabella]

- Elevato traffico tra il client ed il server
- Invasivo per il server
- Elevata contesa di risorse nel `tempdb`
- Metodo meno efficiente
- Opportuno se il sistema destinazione non supporta richieste SQL

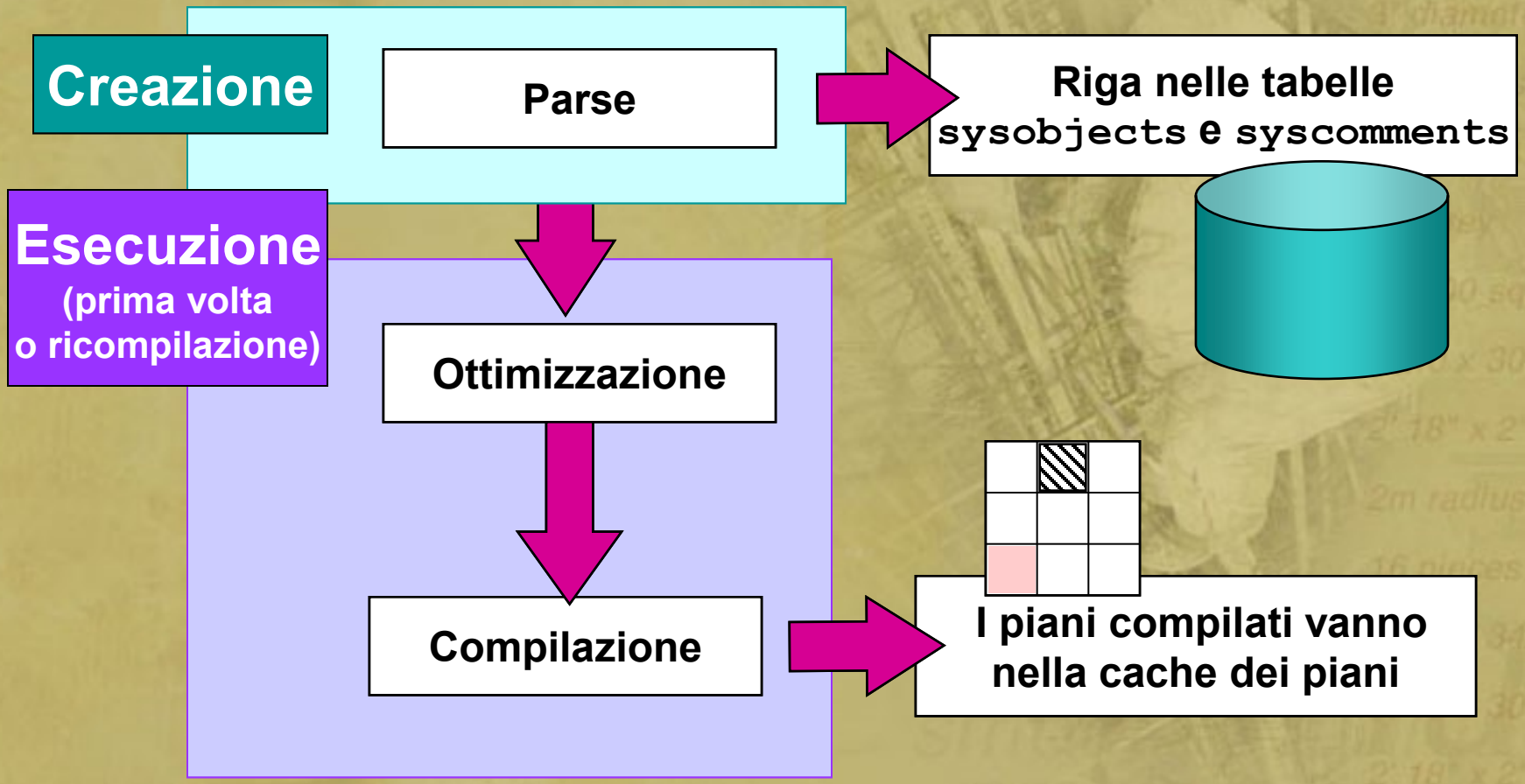
[Come funziona il Query Optimizer]



[Richiesta SQL testuale]

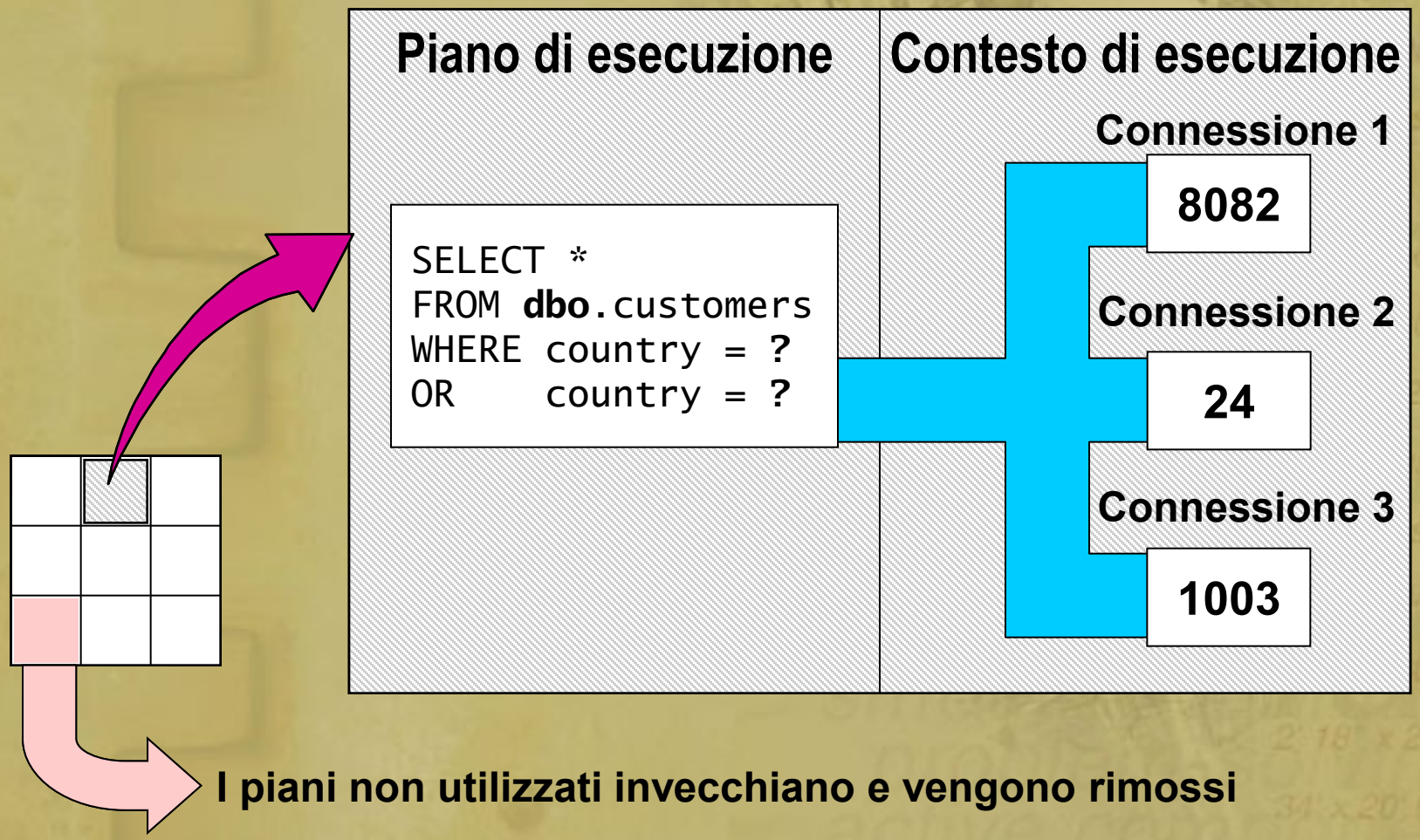
- Demo
- Alta probabilità che il piano di esecuzione non entri in *cache*
- Auto-parametrazione solo per richieste molto semplici
- Uso inefficiente dello spazio dedicato ai piani di esecuzione

[Come funzionano le SP]



[Esecuzioni successive di SP]

Recupero piano di esecuzione



I piani non utilizzati invecchiano e vengono rimossi

[Stored Procedure]

- Demo
- Sono il modo più effciente per accedere a SQL Server!
- Isolano l'applicativo dall'implementazione fisica
- Sicurezza aggiuntiva
- Attenzione alle ri-compilazioni dei piani!

[Ricompilazione SP]

- Quando servono le statistiche di distribuzione dei valori (o un loro aggiornamento)
- Quando avvengono delle modifiche allo schema
- Utilizzo di strutture temporanee
- Alla ripartenza del server
- `DBCC FREEPROCCACHE ;-)`

[Tracciare le SP]

- Sql Server Profiler
  - Stored Procedure: Cache Insert
  - Stored Procedure: ExecContexHit
- Perfomance Monitor (System Monitor)
  - SQL Statistic : SQL Compilations/sec
  - SQL Statistic : SQL Re-Compilations/sec
  - Cache Manager: Cache Hit Ratio (Procedure plans)

[Command e parameters]

- Oggetto `Command` rappresenta il componente ADO per l'esecuzione di *stored procedure* oppure codice SQL ripetuto
- Espone la *collection* (`Parameters`) contenente oggetti di tipo `Parameter` per poter "passare" parametri alla *stored procedure*
- Fondamentale per ottenere risparmio di risorse e quindi prestazioni migliori

[Command e parameters]

- Proprietà principali
  - `ActiveConnection`
    connessione su cui eseguire il comando
  - `CommandText`
    comando da eseguire
  - `Execute`
    esegue il comando e può tornare oggetto di tipo `Recordset`

[Command e parameters]

- Come passare i parametri
  - Richiamare il metodo `Refresh` sulla *collection* `Parameters`
  - Utilizzare metodo `Append` assieme al metodo `CreateParameters`

```
.Parameters.Append .CreateParameter("Country1",
  adWChar , _  adParamInput, 15, Text1.Text)
```

[NoRecordset]

- Nel caso non vi sia la necessità di avere *rowsets* è importante specificare `adExecuteNoRecordset` come opzione al metodo `Execute`

- Ad esempio per operazioni di *update*, *insert* oppure di *lookup*

```
.Execute adExecuteNoRecords
```

[Prepared]

- Proprietà `Prepared = true`
- Permette di “compilare” la *Query* ed eseguirla più volte cambiando solo i parametri
- Surrogato della *stored procedure* lato client
- Simile a `sp_executesql`

[CommandType]

- Specifica il tipo di comando che stiamo per eseguire
- Attenzione agli errori!

```
.CommandType = adCmdStoredProc
```

[Recordset]

- Rappresenta un insieme di record come risultato di una *query*
- E' un cursore a tutti gli effetti
- Può essere implementato lato client / lato server
  - `CursorLocation = adUseClient  (3)`
  - `CursorLocation = adUseServer  (2)`

[Tipi di recordset]

- Come cursore sui dati può avere caratteristiche diverse che influenzano la “visibilità’” e la modalità di “scorrimento” sui record referenziati

```
CursorType=
```

- `adOpenForwardOnly`
- `adOpenStatic`
- `adOpenKeyset`
- `adOpenDynamic`

[Forward Only (`adOpenForwardOnly`)]

- Permette di scorrere i record solo in “avanti”
  - `MoveNext`
  - No `MoveFirst`, `MoveLast` e `MovePrevious`
- Non è sensibile ai dati modificati dalle operazioni di *update*, *insert* e *delete* effettuate da altri client
- Molto veloce
- Utilizza poche risorse

[Static (`adOpenStatic`)]

- Simile al “Forward Only” ma permette lo scorrimento dei record
- Non è sensibile ai dati modificati dalle operazioni di *update*, *insert* e *delete* effettuate da altri client
- Utilizza poche risorse

[KeySet (`adOpenKeyset`)]

- Memorizza solo le chiavi (KeySet)
- Durante lo scorrimento/aggiornamento dei dati utilizza il “KeySet” per localizzare i record
- Il “KeySet” è statico
- E’ sensibile alle operazioni di *update* e *delete* di altri client (dipende dal provider)
- Medio utilizzo di risorse

[Dynamic Cursor (`adOpenDynamic`)]

- E' simile al cursore "KeySet"
- Verifica l'esistenza di nuovi record durante lo scorrimento/aggiornamento
- E' sensibile anche ai dati inseriti
- Alto utilizzo di risorse

[Posizione cursori]

- L'unico tipo che possiamo scegliere dove implementarlo è il cursore *Static*
- ADO non da' errore ma corregge il tipo di cursore per noi
- Al metodo `Close` ri-sistema il tipo di cursore

[Tipi di Lock]

- `AdLockReadOnly` = dati solo lettura no *lock*
- `AdLockOptimistic` = dati bloccati solo da *update*
- `AdLockPessimistic` = dati bloccati solo durante operazioni di *Edit*
- `AdLockBatchOptimistic` = nessun blocco, record disconessi
- Demo

[RecordSet disconessi]

- Permettono di avere un oggetto `Recordset` indipendente dalla connessione

```
With AdoRs
        .CursorLocation = adUseClient
        .CursorType = adOpenStatic
        .LockType = adLockBatchOptimistic
        .ActiveConnection = AdoCnn
        .Open STRSQL
        .ActiveConnection = Nothing
End With
```

[RecordSet disconessi]

- Per consolidare le modifiche apro nuovamente la connessione e chiamo il metodo `UpdateBatch`

```
With AdoRs
        .ActiveConnection = AdoCnn
        .UpdateBatch
        Set .ActiveConnection = Nothing
    End With
```

- Per gestire i conflitti

```
.Filter = adFilterConflictingRecords
```

[Risorse]

- **User Groups**

  http://www.ugiss.org
  http://www.sqlpass.org
  http://www.ugidotnet.org

- **Microsoft**

  http://www.microsoft.com/sql
  http://www.microsoft.com/italy/sql
  http://msdn.microsoft.com/sql
  http://www.microsoft.com/TechNet/prodtechnol/sql

[Risorse]

- **Newsgroups**
  - news://news.ugiss.org

    ugiss.public.ita.*
  - news://msnews.microsoft.com

    microsoft.public.it.sql
    microsoft.public.sqlserver.*
  - Non vi fanno uscire via NNTP ?

    http://msdn.microsoft.com/newsgroups
  - Novità!

    http://www.microsoft.com/italy/sql/support/newsgroup/

[Domande & Risposte]

[GRAZIE!]

PASS
PROFESSIONAL ASSOCIATION FOR SQL SERVER